

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра программного обеспечения информационных технологий

Дисциплина
«Объектно-ориентированные технологии программирования и
стандарты проектирования»

ОТЧЁТ
к лабораторной работе №2

РАБОТА С ОБЪЕКТАМИ
В РАМКАХ ПРОЕКТА «NINJA-ARENA»

Студент группы
№351001
Ушаков А.Д.

Преподаватель
Деменковец Д. В.

Минск 2025

СОДЕРЖАНИЕ

Введение	3
1 Теоретические сведения	4
1.1 Теория и определения из лекционного курса	4
1.2 Ответы на вопросы преподавателя	4
2 Реализация.....	8
2.1 Создание и изменение объектов	8
2.2 Хранение данных	10
2.3 Формирование динамической формы для работы с объектами.....	11
2.5 UI (edit_page.html)	18
3 Тестирование	21
3.1 Добавление персонажа	21
3.2 Изменение персонажа	22
Заключение	24
Список использованных источников	25

ВВЕДЕНИЕ

В рамках данной лабораторной работы реализована система для динамического добавления и редактирования объектов иерархии ниндзя, наследующихся от абстрактного класса `Ninja`. Основной задачей было обеспечить удобное управление этими объектами через графический интерфейс с последующим сохранением изменений в локальном хранилище браузера.

Ключевым элементом системы стала динамическая форма, которая генерируется на основе структуры классов, описанной в JSON-файле. При выборе конкретного класса ниндзя форма автоматически обновляется, отображая только релевантные поля для редактирования. Такой подход исключает необходимость создания отдельных форм для каждого типа объекта и значительно упрощает взаимодействие пользователя с системой.

Все изменения данных сохраняются в `localStorage`, что обеспечивает их доступность между сессиями работы. Особенностью реализации является использование рефлексии для динамического создания объектов — это позволяет системе работать с любыми классами ниндзя без необходимости внесения изменений в основной код при расширении иерархии.

Стартовый набор объектов загружается из backend-части, написанной на TypeScript, что обеспечивает строгую типизацию и надёжность работы. Интерфейс взаимодействия между frontend и backend построен таким образом, чтобы минимизировать нагрузку на сервер и сохранить высокую скорость работы приложения даже при большом количестве данных.

Система автоматически обрабатывает конфликты имён при добавлении новых объектов, заменяя существующие записи при необходимости. Также реализована возможность редактирования ранее созданных ниндзя через специальный режим, при котором форма предварительно заполняется сохранёнными значениями.

В результате был создан удобный и функциональный инструмент для управления иерархией ниндзя, который демонстрирует эффективное сочетание динамических форм, локального хранения данных и рефлексии. Подобный подход может быть успешно применён и для других задач, требующих работы со сложными иерархиями объектов в веб-приложениях.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Теория и определения из лекционного курса

Данные материалы представлены в конспекте лекций, который будет предоставлен во время защиты лабораторной работы.

1.2 Ответы на вопросы преподавателя

Перегрузка. Информация, касающаяся перегрузок, представлена на рисунке 1.2.1.

1) Перегрузка (Overloading)

Перегрузка — это определение нескольких методов с одним именем, но разными параметрами (разное количество или типы аргументов).

◆ C#

```
csharp
CopyEdit
class Example {
    public void Print(int value) {
        Console.WriteLine("Int: " + value);
    }

    public void Print(string value) {
        Console.WriteLine("String: " + value);
    }
}
```

✦ В C# перегрузка работает на этапе компиляции.

◆ TypeScript В TypeScript перегрузка делается через перегруженные сигнатуры.

```
typescript
CopyEdit
class Example {
    print(value: number): void;
    print(value: string): void;
    print(value: any): void {
        console.log(value);
    }
}
```

✦ В TypeScript перегрузка работает на уровне типов, но при компиляции остаётся только одно универсальное определение метода.

Рисунок 1.2.1 – Перегрузка в C# и TypeScript

Переопределение: теоретические сведения представлены на рисунках 1.2.2 и 1.2.3.

2

Переопределение (Overriding)

Переопределение — это замена метода родительского класса в производном классе.

- С# (используется `virtual` и `override`)**

```

csharp
CopyEdit
class Parent {
    public virtual void Show() {
        Console.WriteLine("Parent");
    }
}

class Child : Parent {
    public override void Show() {
        Console.WriteLine("Child");
    }
}
                    
```

✦ В С# переопределение работает через виртуальную таблицу методов (VMT).
- TypeScript (используется `extends`)**

```

typescript
CopyEdit
class Parent {
    show(): void {
        console.log("Parent");
    }
}

class Child extends Parent {
    show(): void {
        console.log("Child");
    }
}
                    
```

✦ В TypeScript переопределение реализуется через прототипное наследование.

Рисунок 1.2.2 – Переопределение в С# и TypeScript

◆ Что можно и нельзя переопределять		
Функция	С# (можно?)	TypeScript (можно?)
Методы (method())	✓ virtual → override	✓ extends
Свойства (property { get; set; })	✓ virtual → override	✓ extends
Статические методы (static method())	✗ Нельзя	✗ Нельзя
Конструкторы (constructor())	✗ Нельзя, но можно base()	✓ Можно через super()
Операторы (+, -, ==)	✓ Можно перегрузить	✗ Нельзя
Индексаторы (this[index])	✓ Можно virtual → override	✗ Нельзя

Рисунок 1.2.3 – Возможности переопределения

Индексаторы. Информация, касающаяся перегрузок, представлена на рисунке 1.2.4.

3 Индексаторы (Indexers)

Индексатор позволяет обращаться к элементам объекта как к массиву (`obj[index]`).

◆ C#

```
csharp
CopyEdit
class Example {
    private string[] data = new string[5];

    public string this[int index] {
        get { return data[index]; }
        set { data[index] = value; }
    }
}
```

✦ В C# индексаторы можно **переопределять** (`override`), но перегружать нельзя.

◆ **TypeScript** В TypeScript аналог индексатора — это **динамический доступ через Record или Map**.

```
typescript
CopyEdit
class Example {
    data: Record<number, string> = {};

    get(index: number): string {
        return this.data[index];
    }

    set(index: number, value: string): void {
        this.data[index] = value;
    }
}
```

✦ В TypeScript индексаторы **нельзя переопределять** в классах.

Рисунок 1.2.4 – Индексаторы в C# и TypeScript

Устройство объектов в C# и TypeScript показано на рисунке 1.2.5.

◆ Как объекты устроены в памяти

◆ C#

1. Объект создаётся в куче (Heap), а переменная-ссылка хранится в стеке (Stack).
2. Объект содержит указатель на таблицу виртуальных методов (VMT).
3. Методы ищутся через VMT.

Пример памяти в C#:

```
makefile
CopyEdit
Stack:
  obj → 0x1234 (указатель на объект)

Heap:
  0x1234 → [VMT, data]
```

◆ TypeScript

1. Объекты создаются в куче, ссылки хранятся в стеке.
2. Методы хранятся в `__proto__` (цепочка прототипов).
3. Поиск идёт через `__proto__`.

Пример памяти в TypeScript:

```
yaml
CopyEdit
Stack:
  obj → 0x1234 (указатель на объект)

Heap:
  0x1234 → { data, __proto__: ParentClass.prototype }
```

✦ В C# VMT делает вызовы быстрыми, а в TypeScript поиск через `__proto__` медленнее.

Рисунок 1.2.5 – Устройство объектов в C# и Typescript

Кратко о рантайме и компиляции на рисунке 1.2.6.

◆ Рантайм и компиляция		
Вопрос	C#	TypeScript
Когда код проверяется?	На этапе компиляции (JIT в .NET)	Только на этапе исполнения (JavaScript)
Есть ли типизация?	✔ Жёсткая (int, string)	✔ Жёсткая, но стирается
Работает ли без компиляции?	✘ Нет (нужен компилятор .NET)	✔ Да (JS-движок)

Рисунок 1.2.6 – Рантайм и компиляция

7

2 РЕАЛИЗАЦИЯ

2.1 Создание и изменение объектов

Для создания объектов была задействована рефлексия [1] – это способность компьютерной программы изучать и модифицировать свою структуру и поведение (значения, метаданные, свойства и функции) во время выполнения. Простым языком: она позволяет вам вызывать методы объектов, создавать новые объекты, модифицировать их, даже не зная имён интерфейсов, полей, методов во время компиляции. Из-за такой природы рефлексии её труднее реализовать в статически типизированных языках, поскольку ошибки типизации возникают во время компиляции, а не исполнения программы (подробнее об этом здесь). Тем не менее, она возможна, ведь такие языки, как Java, C#, TypeScript и другие допускают использование рефлексии.

В файле `ninjaFactory.ts` импортируются существующие типы данных:

```
import { Ninja } from "../ninjas/ninja";
import { fireNinja } from "../ninjas/fire_ninja/fire_ninja";
import { eyesClanNinja } from "../ninjas/fire_ninja/eyes_clan";
.....
```

Далее все типы данных помещаются в глобальный контекст:

```
(globalThis as any).fireNinja = fireNinja;
(globalThis as any).eyesClanNinja = eyesClanNinja;
(globalThis as any).boneClanNinja = boneClanNinja;
.....
```

В конечном счёте создаётся функция для динамического создания объектов на основе переданного в неё типа данных:

```
export function createDynamicNinja(className: string, data: any):
Ninja {
    const NinjaClass = (globalThis as any)[className];

    if (!NinjaClass) {
        throw new Error(`Class "${className}" not found.`);
    }

    return new NinjaClass(data);
}
```

Далее в файле `ninjaCreator.ts` определяется функция, обобщающая создание объекта: задаётся связь с интерфейсом (динамической формой), предназначенным для создания экземпляров, полученные объекты добавляются в контейнер для хранения и т.д. :

```
export function createNinja(
  event: Event,
  form: HTMLFormElement,
  container: Array<Ninja>,
  className: string
): void {
  event.preventDefault();
  const formData = new FormData(form);
  const ninjaData: Record<string, any> = {};

  formData.forEach((value, key) => {
    ninjaData[key] = value;
  });

  try {
    const ninja = createDynamicNinja(className, ninjaData);

    const name = ninjaData.name;
    const existingNinjaIndex = container.findIndex(n => n.name ===
name);

    if (existingNinjaIndex !== -1) {
      container.splice(existingNinjaIndex, 1);
      console.log(`Removed existing ninja with name ${name}`);
    }

    console.log('Object was created:', ninja);
    container.push(ninja);

    alert(`Object ${className} created successfully!`);

  } catch (error) {
    console.error('Error while creating object:', error);

    alert("Error while creating an object of a nonexistent or abstract
class." +
      "\nCHOOSE EXISTENT NINJA!");
  }
}
```

2.2 Хранение данных

По результатам лабораторной работы №1 объекты создавались на backend-части и помещались в специальный контейнер для хранения. Затем этот контейнер экспортировался на страницу с персонажами.

В текущей реализации этот механизм был модифицирован. Исходный «дефолтный» контейнер по-прежнему экспортируется с backend-части, но как только происходят какие-то изменения, хранение переходит в локальное хранилище браузера. Создан статический класс `LocalStorageAccessor`, который предоставляет статические методы для работы с локальным хранилищем браузера: сериализацию и десериализацию данных:

```
export class LocalStorageAccessor {

  static serializeContainer(container, classData) {
    const serializedContainer = container.map(ninja => {
      const baseData = {
        name: ninja.name,
        health: ninja.health,
        chakra: ninja.chakra,
        rank: ninja.rank,
        organization: ninja.organization,
        village: ninja.village,
        appearance: ninja.appearance,
        arenaView: ninja.arenaView,
      };

      const classInfo = classData[ninja.constructor.name];
      if (classInfo && classInfo.fields) {
        classInfo.fields.forEach(field => {
          baseData[field.name] = ninja[field.name];
        });
      }

      return {
        type: ninja.constructor.name,
        data: baseData,
      };
    });

    console.log('Serialized Container:', serializedContainer);
    localStorage.setItem('Container',
      JSON.stringify(serializedContainer));
  }

  static async deserializeContainer(container) {
    let result;
```

```

    let {createDynamicNinja} = await
import('.././build/models/factories/ninjaFactory.js');
    const savedContainerData =
JSON.parse(localStorage.getItem('Container'));

    console.log('Restored Container:', savedContainerData);

    if (savedContainerData) {
        result = savedContainerData.map(({ type, data }) => { return
createDynamicNinja(type, data) } );
        } else {
            result = container;
        }

        return result;
    }
}

```

2.3 Формирование динамической формы для работы с объектами

Для того, чтобы создать динамическую форму работы с объектами, необходимо где-то хранить информацию о существующих типах данных на backend-части, так как напрямую взаимодействие между веб-страницей и сложной иерархической структурой, созданной на языке TypeScript, невозможно. Структурой данных для хранения выступает `classes.json`:

```

{
  "classes": {
    "AbstractNinja": {
      "fields": [
        { "name": "name", "type": "text", "placeholder": "Name" },
        { "name": "health", "type": "number", "placeholder": "Health",
"range": "from 500 to 3000" },
        { "name": "chakra", "type": "number", "placeholder": "Chakra",
"range": "from 300 to 1500" },
        { "name": "rank", "type": "select", "options": ["A", "B", "C",
"S", "K"], "placeholder": "Rank" },
        { "name": "organization",
"type": "select",
"options": ["none", "akatsuki", "anbu", "sanins",
"orodevils"],
"placeholder": "Organization" },
        { "name": "village",
"type": "select",
"options": [
"Konohagakure",
"Kirigakure",
"Kumogakure",

```

```

        "Iwagakure",
        "Uzushinogakure",
        "Amegakure",
        "Sunagakure",
        "Otogakure"],
        "placeholder": "Village" },
    { "name": "appearance", "type": "text", "placeholder":
"Appearance" },
    { "name": "arena_view", "type": "text", "placeholder": "Arena
View" }
    ],
    "children": ["fireNinja", "lightningNinja", "waterNinja",
"windNinja"]
    },
    "fireNinja": {
        "fields": [
            { "name": "flame_power", "type": "number", "placeholder":
"Flame Power", "range": "from 0.1 to 3" }
        ],
        "children": ["eyesClanNinja", "boneClanNinja"]
    },

```

Функции для создания формы находятся в `form_creator.js`:

```
import {createNinja} from "../../build/components/ninjaCreator.js";
import {LocalStorageAccessor} from "../localStorageAccessor.js";

let classData = {};

export function processNinjasData() {

function processJSONData(JSONData) {
    classData = JSONData.classes;
    addClasses('AbstractNinja', document.getElementById('classSelector'));
    updateForm('AbstractNinja');
}

function fetchData(pathToJSON) {
    fetch(pathToJSON)
        .then( response => response.json() )
        .then( data => processJSONData(data) )
        .catch( error => console.error('Ошибка загрузки JSON:', error) );
}

fetchData(`classes.json`);
}

function addClasses(className, selector, level = 0) {

function fillClassAsOption() {
    const option = document.createElement('option');

    option.value = className;
    option.innerHTML = '    '.repeat(level * 2) + className;
```

```

    selector.appendChild(option);
  }

  function addChildClasses() {
    if (classInfo.children) {
      classInfo.children.forEach(child => {
        addClasses(child, selector, level + 1);
      });
    }
  }

  const classInfo = classData[className];

  if (classInfo) {
    fillClassAsOption();
    addChildClasses();
  } else {
    console.log('Ошибка при получении данных о классе');
  }
}

export function updateClassSelector() {
  const selector = document.getElementById('classSelector');

  selector.innerHTML = '';
  addClasses('AbstractNinja', selector);
}

export async function updateForm(className, currentContainer) {

  function resetCharacterCard() {
    const card = document.querySelector('.character-card');
    if (!card) return;

    const nameElement = card.querySelector('.character-card__name');
    if (nameElement) nameElement.textContent = '';

    const propertyElements = card.querySelectorAll('.character-card__field_property');
    for (let i = 0; i < propertyElements.length; i++) {
      const valueElement = propertyElements[i].nextElementSibling;
      if (valueElement && valueElement.classList.contains('character-card__field_value')) {
        valueElement.textContent = '';
      }
    }

    updateFieldValue(card, 'Village', 'Konohagakure');
    updateFieldValue(card, 'Organization', 'none');
    updateFieldValue(card, 'Ninja rank', 'A');
  }

  function addFieldsForChosenClass(form) {
    let currentClass = className;

    while (currentClass) {
      const classInfo = classData[currentClass];

      if (classInfo && classInfo.fields) {

```

```

        classInfo.fields.forEach(field => {
            const input = createField(field);
            form.appendChild(input);
        });
    }

    currentClass = Object.keys(classData).find(key =>
        classData[key].children && classData[key].children.includes(currentClass));
    }
}

function createSubmitButton(submitButton) {
    submitButton.type = 'submit';
    submitButton.textContent = 'Add Changes';
}

function createForm(form) {
    const submitButton = document.createElement('button');
    createSubmitButton(submitButton);
    form.appendChild(submitButton);
}

function createFormContainer(formContainer) {
    formContainer.innerHTML = '';

    const form = document.createElement('form');
    form.classList.add('form-grid');

    addFieldsForChosenClass(form);
    createForm(form);
    formContainer.appendChild(form);

    const classSelector = document.getElementById('classSelector');
    const urlParams = new URLSearchParams(window.location.search);
    const isEditMode = urlParams.get('source') === 'card';

    if (isEditMode && classSelector) {
        classSelector.disabled = true;
    }

    form.addEventListener('submit', function (event) {
        classSelector.disabled = false;

        createNinja(event, this, currentContainer, className);
        console.log(`Current container include: ${currentContainer}`);

        LocalStorageAccessor.serializeContainer(currentContainer, classData);

        if (isEditMode && classSelector) {
            setTimeout(() => { classSelector.disabled = true; }, 0);
        }
    });
}

resetCharacterCard();

currentContainer = await
LocalStorageAccessor.deserializeContainer(currentContainer);

const formContainer =
    document.getElementById('formContainer');

```

```

    createFormContainer(formContainer)
}

function createField(field) {

    function updateCharacterCard(fieldName, value) {
        const card = document.querySelector('.character-card');
        if (!card) return;

        switch(fieldName) {
            case 'name':
                const nameElement = card.querySelector('.character-card__name');
                if (nameElement) nameElement.textContent = value;
                break;
            case 'health':
                updateFieldValue(card, 'Health', value);
                break;
            case 'chakra':
                updateFieldValue(card, 'Chakra', value);
                break;
            case 'rank':
                updateFieldValue(card, 'Ninja rank', value);
                break;
            case 'organization':
                updateFieldValue(card, 'Organization', value);
                break;
            case 'village':
                updateFieldValue(card, 'Village', value);
                break;
            case 'appearance':
                const imgElement = card.querySelector('.character-card__img');
                if (imgElement) {
                    imgElement.src = value;
                    imgElement.onerror = function() {
                        this.style.display = 'none';
                    };
                    imgElement.onload = function() {
                        this.style.display = '';
                    };
                }
                break;
        }
    }

    function createLabel(subField) {
        const label = document.createElement('label');
        label.textContent = field.placeholder;
        subField.appendChild(label);
    }

    function createInput(subField) {
        const input = document.createElement('input');

        input.type = field.type;
        input.name = field.name;
        input.placeholder = field.range ? field.range : field.placeholder;

        const urlParams = new URLSearchParams(window.location.search);
        const source = urlParams.get('source');
        if (field.name === 'name' && source === 'card') {
            input.readOnly = true;
        }
    }
}

```

```

    }

    input.addEventListener('input', function() {
        updateCharacterCard(field.name, this.value);
    });

    subField.appendChild(input);
}

function createOption(select, option) {
    const optionElement =
        document.createElement('option');

    optionElement.value = option;
    optionElement.textContent = option;
    select.appendChild(optionElement);
}

function createSelect(subField) {
    const select = document.createElement('select');
    select.name = field.name;

    field.options.forEach(option => {
        createOption(select, option);
    });

    select.addEventListener('change', function() {
        updateCharacterCard(field.name, this.value);
    });

    subField.appendChild(select);
}

function createFileInput(subField) {
    const container = document.createElement('div');
    container.classList.add('file-input');

    const textInput = document.createElement('input');
    textInput.type = 'text';
    textInput.classList.add('file-input__text');
    textInput.name = field.name;
    textInput.placeholder = field.placeholder;
    textInput.readOnly = true;

    const fileInput = document.createElement('input');
    fileInput.type = 'file';
    fileInput.classList.add('file-input__file');
    fileInput.name = field.name;
    fileInput.accept = 'image/*';

    fileInput.addEventListener('change', function () {
        if (this.files && this.files[0]) {
            const filePath =
`../../../../../src/assets/edit_page/edit_pictures/${this.files[0].name}`;

            textInput.value = filePath;
            console.log(textInput.value);

            updateCharacterCard(field.name, filePath);
        }
    });
}

```

```

    });

    container.appendChild(fileInput);
    container.appendChild(textInput);

    subField.appendChild(container);

    fileInput.addEventListener('click', function () {
        const cardImage = document.querySelector('.character-card__img');

        fileInput.files = new DataTransfer().files;
        textInput.value = '';
        cardImage.src = '';
    });
}

const result = document.createElement('div');

if (field.name === 'appearance' || field.name === 'arena_view') {
    createLabel(result);
    createFileInput(result);
} else if (field.type === 'select') {
    createLabel(result);
    createSelect(result);
} else {
    createLabel(result);
    createInput(result);
}

return result;
}

function updateFieldValue(card, propertyName, value) {
    const propertyElements = card.querySelectorAll('.character-card__field_property');

    for (let i = 0; i < propertyElements.length; i++) {

        if ( propertyElements[i].textContent.trim() === propertyName ) {
            const valueElement = propertyElements[i].nextElementSibling;

            if ( valueElement && valueElement.classList.contains('character-card__field_value') ) {
                valueElement.textContent = value;
            }

            break;
        }
    }
}

export function fillFormFromSessionData(characterData) {

    function setNinjaClass(ninjaType) {
        const classSelector = document.getElementById('classSelector');
        classSelector.value = ninjaType;
        classSelector.dispatchEvent(new Event('change'));
    }

    function updateImageCard(imagePath) {
        const imgElement = document.querySelector('.character-card__img');
        if (!imgElement) return;
    }
}

```

```

    imgElement.src = imagePath;
    imgElement.onerror = () => imgElement.style.display = 'none';
    imgElement.onload = () => imgElement.style.display = '';
  }

  function fillFileInput(form, value) {
    const textInput = form.querySelector('.file-input__text');
    if (textInput) {
      textInput.value = value;
      textInput.dispatchEvent(new Event('input'));
    }
  }

  function fillFormField(input, value) {
    input.value = value;
    const eventType = input.tagName === 'SELECT' ? 'change' : 'input';
    input.dispatchEvent(new Event(eventType));
  }

  setNinjaClass(characterData.ninjaType);

  setTimeout(() => {
    const form = document.querySelector('.form-grid');
    if (!form) return;

    Object.entries(characterData).forEach(([key, value]) => {
      if (key === 'ninjaType') return;

      const input = form.querySelector(`[name="${key}"]`);
      if (!input) return;

      if (key === 'appearance') {
        updateImageCard(value);
        fillFileInput(form, value);
        return;
      }

      fillFormField(input, value);
    });

    sessionStorage.removeItem('editCharacterData');
  }, 100);
}

```

2.5 UI (edit_page.html)

Форма представлена на странице `edit_page.html`, к которой подключён `edit_page.js`. В нём импортируется «дефолтный» контейнер с backend-части, а также подключаются обработчики событий и функции из `form_creator.js`:

```

import {Container} from "../../build/components/container.js";
import {processNinjasData, updateClassSelector, updateForm,
fillFormFromSessionData} from "../form_creator.js";

processNinjasData();

```

```

document.getElementById('classSelector')
  .addEventListener('change', async function () {
    const selectedOption = this.options[this.selectedIndex];

    this.options[this.selectedIndex].textContent =
      selectedOption.textContent.trim();
    await updateForm(this.value, Container);
  });

document.getElementById('classSelector')
  .addEventListener('mousedown', function () {
    updateClassSelector();
  });

window.addEventListener('load', () => {

  setTimeout(() => {
    const editData = sessionStorage.getItem('editCharacterData');
    if (editData) {
      const characterData = JSON.parse(editData);
      fillFormFromSessionData(characterData);
    }

    const container_left =
      document.getElementsByClassName('container_left')[0];
    const container_right =
      document.getElementsByClassName('container_right')[0];

    if (container_left && container_right) {
      container_left.style.height =
        window.getComputedStyle(container_right).height;
    }
  }, 100);
});

```

Переход на страницу `edit_page.html` осуществляется двумя способами: через стартовую страницу (`start_page.html`, кнопка *New*, рис. 2.5.1) или при нажатии на карточку персонажа для редактирования (рис. 2.5.2).

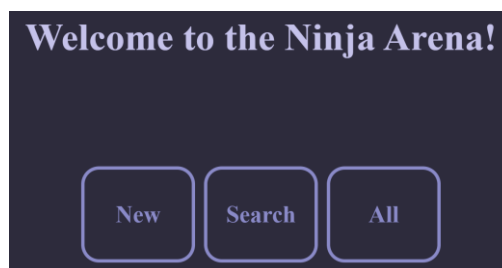


Рисунок 2.5.1 – Стартовая страница

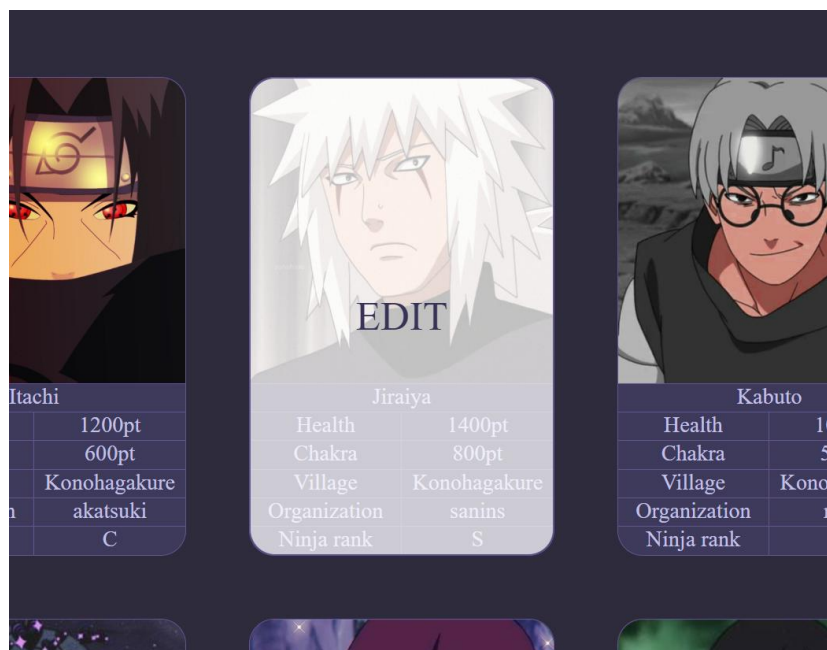


Рисунок 2.5.2 – Редактирование персонажа

Внешний вид формы на edit_page.html представлен на рисунке 2.5.3:

Create or Edit a Ninja



Konan	
Health	1300
Chakra	400
Village	Amegakure
Organization	akatsuki
Ninja rank	C

Select Ninja Type:
boneClanNinja

Bone Count
from 100 to 1000

Flame Power
from 0.1 to 3

Health
1300

Rank
C

Village
Amegakure

Arena View
Arena View

Cursed Mark
No

Name
Konan

Chakra
400

Organization
akatsuki

Appearance
../components/characters/konan/images/kor

Add Changes

Рисунок 2.5.3 – Динамическая форма

3 ТЕСТИРОВАНИЕ

3.1 Добавление персонажа

На рисунках 3.1.1 – 3.1.4 показана последовательность успешного прохождения теста.

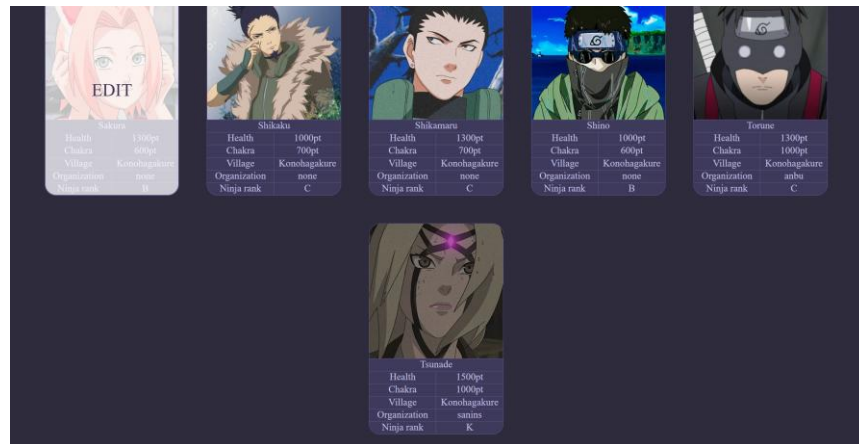


Рисунок 3.1.1 – Начальное состояние контейнера

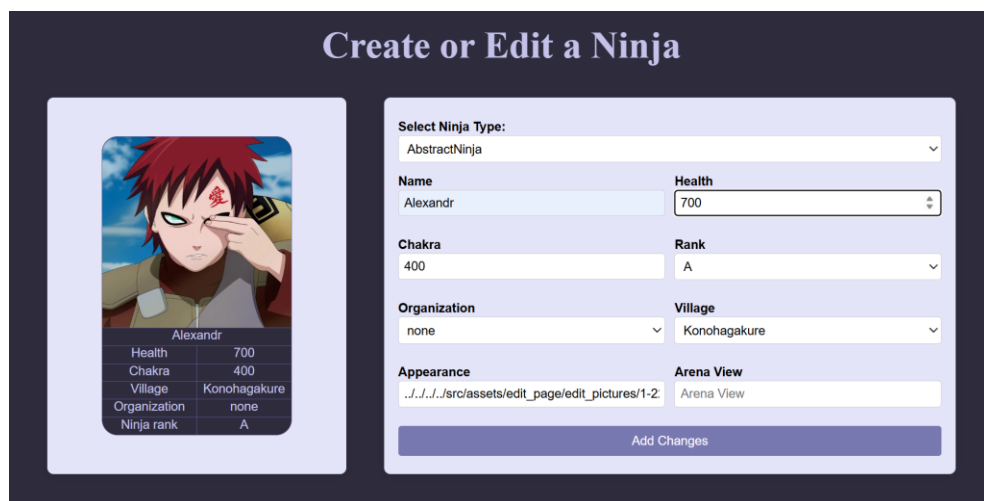


Рисунок 3.1.2 – Создание объекта

Стоит отметить, что после выхода из браузера и даже перезагрузки ОС данные не исчезают, а продолжают существовать в локальном хранилище браузера до тех пор, пока пользователь их самостоятельно не очистит.

```

▶ 10: {type: "militaryDoctor",...}
▶ 11: {type: "shadowGhost",...}
▶ 12: {type: "shadowMaster",...}
▶ 13: {type: "antKing",...}
▶ 14: {type: "antKing",...}
▶ 15: {type: "neurosurgeon",...}
▼ 16: {type: "devilEyeNinja",...}
  ▼ data: {name: "Alexandr", health: "700", chakra: "400", rank: "A", organization: "none",...}
    appearance: ""
    arenaView: ""
    chakra: "400"
    health: "700"
    name: "Alexandr"
    organization: "none"
    rank: "A"
    village: "Konohagakure"
    type: "devilEyeNinja"

```

Рисунок 3.1.3 – Появление объекта в локальном хранилище

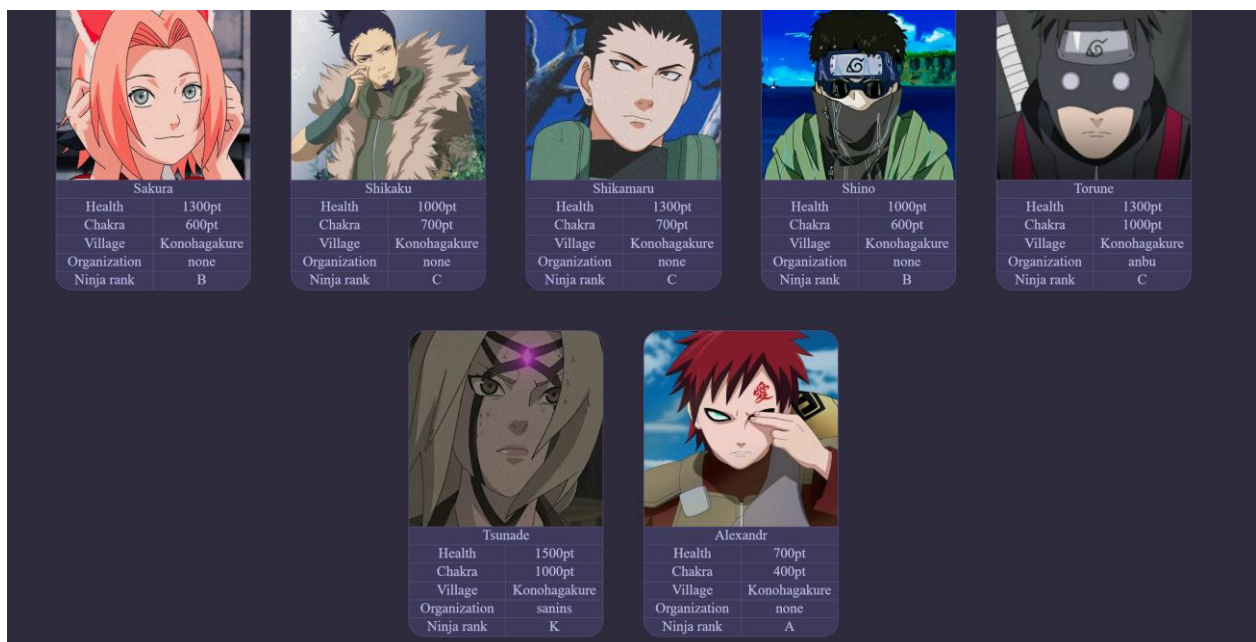


Рисунок 3.1.4 – Конечное состояние контейнера

3.2 Изменение персонажа

Изменим персонажа из теста 3.1. Его параметр chakra изначально равен 400pt. Изменим на 500pt.

Сначала выберем персонажа на container_page.html, затем перейдём на форму редактирования и внесём изменения (см. рисунки 3.2.1 – 3.2.3 на следующей странице).



Рисунок 3.2.1 – Выбор персонажа для изменения

Create or Edit a Ninja



Alexandr	
Health	700
Chakra	500
Village	Konohagakure
Organization	none
Ninja rank	A

Select Ninja Type:

Eye Level: Flame Power:

Name: Health:

Chakra: Rank:

Organization: Village:

Appearance: Arena View:

Рисунок 3.2.2 – Редактирование

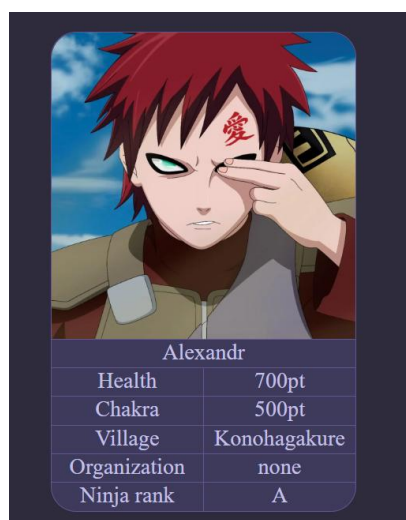


Рисунок 3.2.3 – Изменённый персонаж (успешное завершение теста)

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была успешно реализована система для динамического управления иерархией объектов ниндзя, демонстрирующая ключевые принципы объектно-ориентированного программирования (ООП). Основные аспекты ООП, которые были применены в проекте, включают рефлексия, фабричные методы, наследование, полиморфизм и инкапсуляцию.

Рефлексия стала одним из центральных механизмов работы системы. Она позволила динамически создавать объекты на основе их имён классов, описанных в JSON-файле, что обеспечило гибкость и расширяемость кода. Благодаря этому добавление новых типов ниндзя не требовало изменений в основной логике программы. Рефлексия также использовалась для доступа к свойствам и методам объектов во время выполнения, что было критически важно для работы динамических форм.

Фабричный метод [2] был реализован в модуле `ninjaFactory.ts`, где создание объектов инкапсулировано в функции `createDynamicNinja`. Этот подход позволил централизовать логику создания объектов и упростить их управление. Фабрика обеспечила единую точку для генерации экземпляров классов, что сделало код более читаемым и поддерживаемым.

Наследование и полиморфизм проявились в построении иерархии классов ниндзя. Абстрактный класс `Ninja` выступил в роли базового, от которого наследовались специализированные классы, такие как `fireNinja` и `eyesClanNinja`. Это позволило переопределять методы и свойства в дочерних классах, сохраняя при этом единый интерфейс для работы с разными типами объектов. Такой подход упростил добавление новых видов ниндзя и обеспечил согласованность поведения объектов.

Инкапсуляция была реализована через чёткое разделение данных и методов их обработки. Например, класс `LocalStorageAccessor` инкапсулировал логику работы с локальным хранилищем, предоставляя статические методы для сериализации и десериализации данных. Это улучшило безопасность кода и снизило вероятность ошибок при работе с хранилищем.

Динамические формы, генерируемые на основе JSON-описания классов, продемонстрировали принцип открытости/закрытости (Open/Closed Principle). Система легко расширялась новыми типами объектов без изменения существующего кода, что особенно важно для масштабируемости проекта.

Использование `localStorage` для сохранения данных между сеансами работы подчеркнуло важность устойчивости системы и её независимости от внешних серверов. Это решение обеспечило сохранность данных даже после перезагрузки браузера или операционной системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Tproger [Электронный ресурс]. – Режим доступа :
<https://tproger.ru/translations/programming-concepts-introspection-reflection>.

[2] Refactoring Guru [Электронный ресурс]. – Режим доступа :
<https://refactoring.guru/ru/design-patterns/factory-method>.